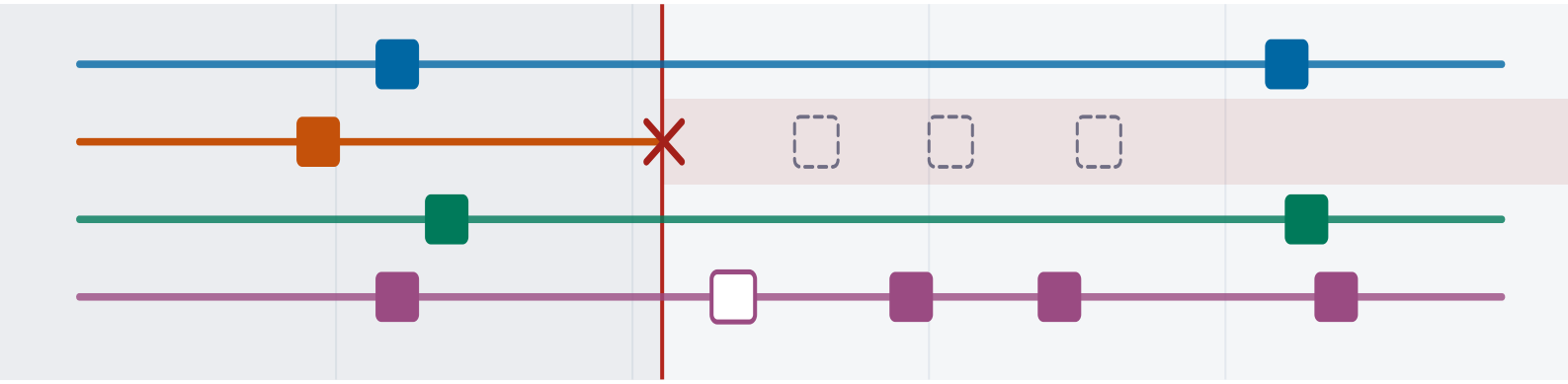
DeTour

Replans a delivery fleet from where it actually is, and arranges a rescue with a person in charge.



Team ID	to be filled in
Team name	to be filled in
Problem statement	**S-01**, ReRoute: Dynamic Fleet & Delivery Replanning
Domain	4, Supply Chain & Optimization
Prototype	github.com/shreyaaassss/Detour

DeTour assigns deliveries to vehicles within capacity and time windows, then replans from the state of the day when a road slows, a vehicle fails or an urgent delivery arrives. Every change is checked by a separate validator and explained delivery by delivery. For a breakdown, a stranded driver reports by voice in their own language, the head of logistics approves a ranked shortlist, a nearby driver is offered the job in the app, and if they do not answer within 60 seconds an automated voice agent speaks to them live on their phone.

1. Problem understanding

1.1 The situation

Each morning a dispatcher decides which vehicle carries which deliveries, and in what order. Vehicles have capacities, deliveries have sizes and time windows, and travel times vary. The plan rarely survives the day: a bridge closes, a vehicle breaks down, an urgent delivery arrives. Today the dispatcher reacts with a spreadsheet and experience. An unchecked change can overload a vehicle, miss a window, or undo a stop that was already completed. And someone must decide what to tell each affected customer.

1.2 What problem S-01 asks for, and how we read it

The problem statement requires	Our reading
Assign deliveries to vehicles within capacity and time windows, with an explainable algorithm whose limits are stated. Show arrival times, including waiting.	A valid plan with arrival, wait, service start, departure and load at every stop. The heuristic and its limits are stated in the interface.
Never overload a vehicle at any point. Report an unplaceable delivery with the reason.	Capacity is checked at every stop, including pickups. Every undeliverable parcel carries a specific reason.
Three disruptions: a road closure for a stated period, a vehicle failure at a stated time, an urgent delivery added mid-day. Structured or described in text.	All three are supported, entered as a form or described in free text. A driver can also report a breakdown by voice. Text and voice are interpreted into the same structures.
A dispatcher must confirm interpreted text before it takes effect. An unknown location is rejected, not guessed.	Interpretation only ever produces a preview. A place that is not on the known list is refused, and the refusal is logged.
Replan from the current state: completed stops stay fixed, and a failed vehicle receives no stops after its failure time.	The state of every vehicle at time T is rebuilt. Completed stops are frozen, and a separate validator checks the result.
Show original and revised plans side by side, each delivery kept, moved or broken, with a reason.	A comparison view with a plain-language reason for every delivery, written so it can go into a customer message.

The booklet's common rules also apply: a working MVP that judges can test live, reproducible inputs, meaningful AI, explicit checkable rules where decisions have consequences, fictional data only, and a core that does not depend on external services.

1.3 What makes it hard

A replan starts mid-day. It is not a fresh plan. Completed stops cannot change, a stop may be in progress, and a vehicle may be part-way along a leg. A tool that re-solves from scratch can produce a plan that undoes the morning.

Parcels are physical. A parcel already loaded on a running vehicle cannot simply be handed to another vehicle on paper. Modelling that honestly changes what a replan is allowed to do.

A breakdown strands parcels, and someone has to agree to fetch them. The rescue needs a pickup at the breakdown site and a handover, and the rescuer's own deliveries must still meet their windows. This is a decision involving several people under time pressure, not only a calculation.

Words are unreliable. A report may be typed or spoken, in Hindi, Marathi or English, and may name a place the system does not know. Guessing would put a vehicle in the wrong place.

The dispatcher has to explain. A new plan is not enough. For every delivery they need to know whether it kept its slot, moved, or cannot be made, and why.

---

2. Proposed solution

DeTour is a deterministic planning engine surrounded by people and AI that help it, never the other way round. The engine decides assignment, feasibility and timing. AI reads and speaks. People approve anything with consequences.

2.1 Four design principles

- **The engine decides, AI assists.** Assignment, timing and every ETA come from code that follows explicit rules. AI transcribes, extracts, speaks and calls. It never assigns anything.
- **Hard constraints never relax.** Capacity, time windows, shift end, closures and frozen stops hold in every plan and every fallback.
- **Humans approve consequential actions.** The head approves an exact shortlist of drivers for an exact version of the plan. If the plan changes afterwards, the approval is void.
- **Degrade gracefully, and record everything.** Every voice or AI step has a typed or button fallback, and every action, refusal, timeout and escalation goes into a tamper-evident audit log.

2.2 The planning engine

The plan is built by **cheapest insertion followed by local search** (relocate, swap, 2-opt). The objective is compared in order: fewest undeliverable parcels, then fewest moved, then least driving time. Travel is time-dependent: a closure lengthens a leg only if the vehicle departs inside the closure period. At every stop the plan records arrival, wait, service start, departure and load.

A vehicle that would arrive early waits at the customer, and the wait is shown. If the wait would exceed 20 minutes it holds back where it is instead, so it is not stranded at a customer and stays available. A parcel that cannot be placed gets one of four specific reasons (too big for any vehicle, window closes before anyone can arrive, every vehicle full, or it would make another delivery late).

Limits, stated in the interface: the method is a heuristic and not provably optimal; travel times come from a simulated matrix; return to the depot is not counted in driving time; local search stops at a local optimum.

2.3 Replanning from the current state

- **What is frozen at time T:** completed stops, the stop in progress, and the next stop of a vehicle that is mid-leg (it cannot be redirected before it arrives).
- **A failed vehicle** receives no stops after its failure time. Its remaining parcels are collected from the breakdown site, with a modelled 5-minute handover that is shared when one rescuer takes several parcels.
- **Parcels aboard a running vehicle stay on it**: delivered by it, or reported broken. Only a failed vehicle's parcels and newly added urgent parcels can change vehicle.
- **A closure** affects only legs that depart after it is reported. It cannot rewrite history.
- **An urgent delivery** is collected from the depot (or a stated place), so a vehicle that has left must come back for it. If it cannot be placed, the reason is given.

2.4 An independent validator

A separate module re-times every route from scratch and checks the result. It works on plain data and imports nothing from the planner (an automated test enforces this), so it cannot repeat the planner's mistakes. It checks that every delivery appears exactly once, arrival and departure times match a recomputation, closures are applied, windows are met, load never exceeds capacity or goes negative, vehicles finish within their shift, pickups come before their deliveries, frozen stops are unchanged, and a failed vehicle has nothing after its failure time. **A plan that fails is never shown as valid, and a rescue that fails is never committed.**

2.5 Explaining every change

Each delivery in the revised plan is classified, with a reason:

Kept same vehicle, window still met; any shift in time is shown. Moved a different vehicle, or the time shifts by 15 minutes or more, or the visiting order changes. Broken cannot be delivered within its window, or at all.

> **Moved.** Moved from V2 (Ravi) to V4 (Meera): V2 broke down at Hadapsar at 14:05. V4 collects the parcel there and delivers at 14:50 (window 14:00-15:00).
> **Broken.** Cannot be delivered within its window (14:30-14:55): window closes at 14:55 but the earliest any vehicle can arrive is 14:57.

From the same facts the system drafts a customer message for each affected delivery. The dispatcher approves it. Drafts are filled in from the plan, contain no internal ids or driver names, and are never sent automatically.

2.6 The breakdown rescue flow

This is the feature that turns a replanning engine into a working emergency procedure. It needs several people to act within minutes, and each step is checked.

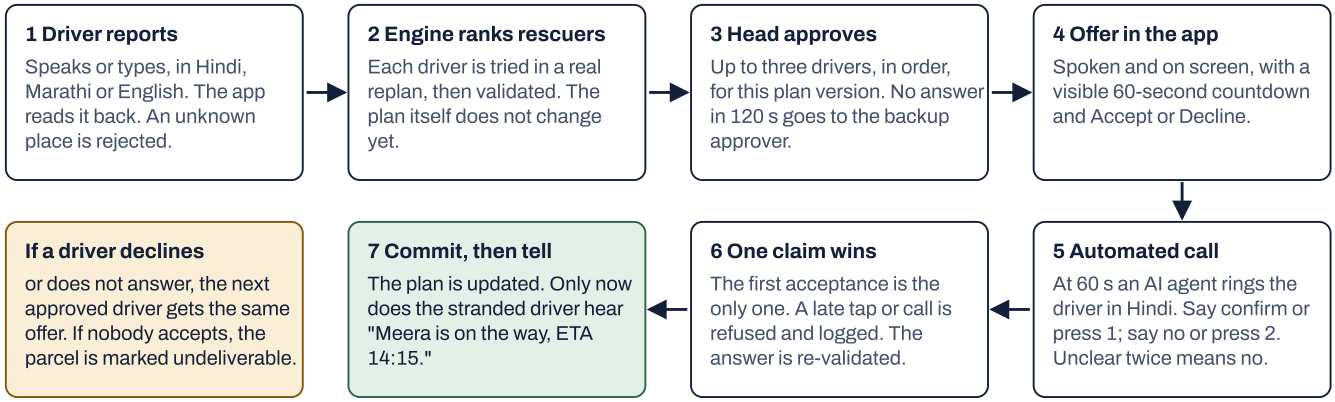


Figure 1. The rescue flow. Steps 1 to 4 change nothing in the plan. The plan changes only at step 7, after a driver has accepted and the result has been re-validated.

Worked example from the prototype

It is 14:05. Van V2 (driver Ravi) breaks down at Hadapsar carrying urgent medicine D7 for Kothrud Clinic, due by 15:00, plus D8 and D9. The engine tries each other driver as the rescuer:

Driver	Reaches the van	Leaves after handover	Delivers D7	Result
Meera (V4)	14:15	14:20	14:50	Feasible, rank 1
Sana (V3)	14:20	14:25	14:55	Feasible, rank 2
Imran (V5)	14:23	14:28	14:58	Feasible, rank 3
Arjun (V1)	14:32	14:37	15:07	Not possible: misses the 15:00 window

The head approves Meera, Sana and Imran. Meera does not tap within 60 seconds, so the agent calls her; she says "confirm". The plan is re-validated and committed. D7 is delivered by V4 at 14:50, D8 moves to V4 as well, and D9 is broken: the earliest anyone could reach Baner is 14:57, two minutes after its window closes.

Rules the rescue flow enforces

- **No early promises.** The stranded driver sees "request sent, awaiting approval" until the assignment is committed.
- **One claim only.** A tap and a call arriving together cannot both win. Closed offers refuse late answers, and the refusal is logged.
- **Re-validate on accept.** Time has passed, so feasibility is checked again before the plan changes. If it fails, the next driver is offered.
- **Approval binds to the shortlist and the plan version.** Any plan change voids it, closes open offers, tells drivers to stand down, and asks the head again with a fresh proposal.
- **No double-booking.** A driver with an open offer for one breakdown is shown as busy for another.
- **Honest failure.** If nobody accepts, the parcel is reported undeliverable. The engine does not assign it to someone who never agreed.

Timer	Value	If it runs out
Driver confirms the read-back	30 s	The report is re-prompted, then dropped after three tries
Head responds	120 s	The backup approver is asked, then the dispatcher is alerted
App offer	**60 s**	An automated call is placed
Call rings	25 s	Counts as declined; next driver
Call conversation	45 s	Counts as no response; next driver

2.7 A tamper-evident audit log

Every accepted change, refusal, approval, notification, call event, timeout and escalation is recorded with who did it and when. Each entry carries the hash of the one before it, so editing, deleting or reordering any entry is detectable. A log that fails verification is archived and never extended.

3. Intended users

DeTour is built for a small delivery fleet (the demonstration uses five vehicles and twelve deliveries) where one dispatcher plans the day and a head of logistics is responsible for emergencies. The demonstration scenario is a fictional city fleet carrying, among other parcels, urgent medicine for a clinic.

Role	Screen	What they do, and what they need
Dispatcher	Dispatcher console (laptop)	Loads the day, applies or describes disruptions, reviews the effect before applying it, compares original and revised plans, approves customer messages, and reads the audit log. Needs speed and trust: a change they can explain.
Stranded driver	Driver app (phone)	Reports a breakdown by voice or text in their own language, confirms a read-back, and is told what is happening. Needs something they can use roadside, in a language they speak, with no promise made that is not yet true.
Head of logistics, and a backup approver	Head console (phone or laptop)	Receives the alert, sees a ranked list of rescue drivers with times and the impact on other deliveries, and approves up to three. Needs to decide in seconds, with a backup if they cannot.
Rescue driver	Driver app, then a live voice call in the app	Receives a spoken offer with a countdown, and is spoken to by the voice agent if they do not answer. Needs a short, unambiguous request and an easy way to accept or decline.
Customers	Messages drafted by the system	Do not use the system directly. They receive an honest, specific message approved by the dispatcher.

4. AI integration

AI is used where language and speech are the problem. It is not used where a rule or a calculation is the answer. Each use has a guardrail and a fallback, so no step depends on an external service.

Task	Service	Guardrail	Fallback
Read a typed report into a disruption	OpenAI language model (optional)	Fills in a fixed form and nothing else. Every place must be on the known list *and* actually named in the text. Times and numbers it invents are flagged. A strict rule reader cross-checks it; if they disagree, the rule reader wins and the other reading is shown. The dispatcher confirms.	The rule reader alone
Understand a spoken report	ElevenLabs speech-to-text (Hindi, Marathi, English)	The transcript passes the same checks as typed text. An unknown place is rejected. The driver confirms a read-back before anything proceeds. Audio is not stored.	The browser's speech recognition, then typing or buttons
Speak read-backs, offers and updates	ElevenLabs text-to-speech	Speaks only sentences the engine produced, found by reference. It never accepts free text, so it cannot say what the plan does not back. Every sentence is cached, and a per-run character cap limits spend.	The phone's own voice, then text on screen

Escalation call	ElevenLabs conversational agent, in the driver's browser	Opens with the engine's exact script. It is instructed never to add, change or promise a fact. It takes one answer, spoken or by on-screen 1 and 2 buttons (browsers cannot send keypad tones), repeats once, and reports through a tool. Calls are capped per run, and only the driver being asked can open one.	The dispatcher's "simulate the call" buttons, or a typed answer
Customer wording	No model: templates	Filled in from the engine's facts. A generated sentence could be wrong about a time, so none is used.	Not needed

Where AI never acts. It does not assign deliveries, decide feasibility, compute a time or an ETA, approve a rescue, or change the plan. Those belong to the engine, the validator and people. This is what the booklet means by AI that informs a decision but does not silently override it.

5. How this differs from existing approaches

Typical approach	Where it falls short for this problem	What DeTour does instead
Spreadsheet and experience	Fast to start, but nothing checks a change. Capacity and windows are easy to break, and explaining the change is manual.	Every change is validated by an independent module and explained delivery by delivery.
Route planners that solve the whole day again	A new solution can reorder work already done and move parcels that are physically on a van.	Replans from the state at time T: history is frozen, parcels stay where they are, and the objective minimises churn.
Vehicle-tracking dashboards	They show where things are, not what to do next, and not whether a proposed fix is feasible.	It produces the fix, proves it feasible, and shows what it does to every other delivery.
A chatbot or AI that proposes changes	A fluent answer can be confidently wrong about a place or a time, and nothing stops it being applied.	AI only fills in a form. The result is verified against known places, previewed, and applied only on confirmation.
Handling a breakdown by phone calls and messages	No ordering, no timers, no record, and two people can both think they have the job.	A structured procedure: ranked candidates, a human approval, a 60-second offer, an automated call, one claim, and a full audit trail.

Two further points set it apart. It works in the driver's own language, with spoken reports and read-backs, which matters roadside. And the core demonstration needs no paid service: every AI step has a local fallback, as the booklet requires.

6. System architecture

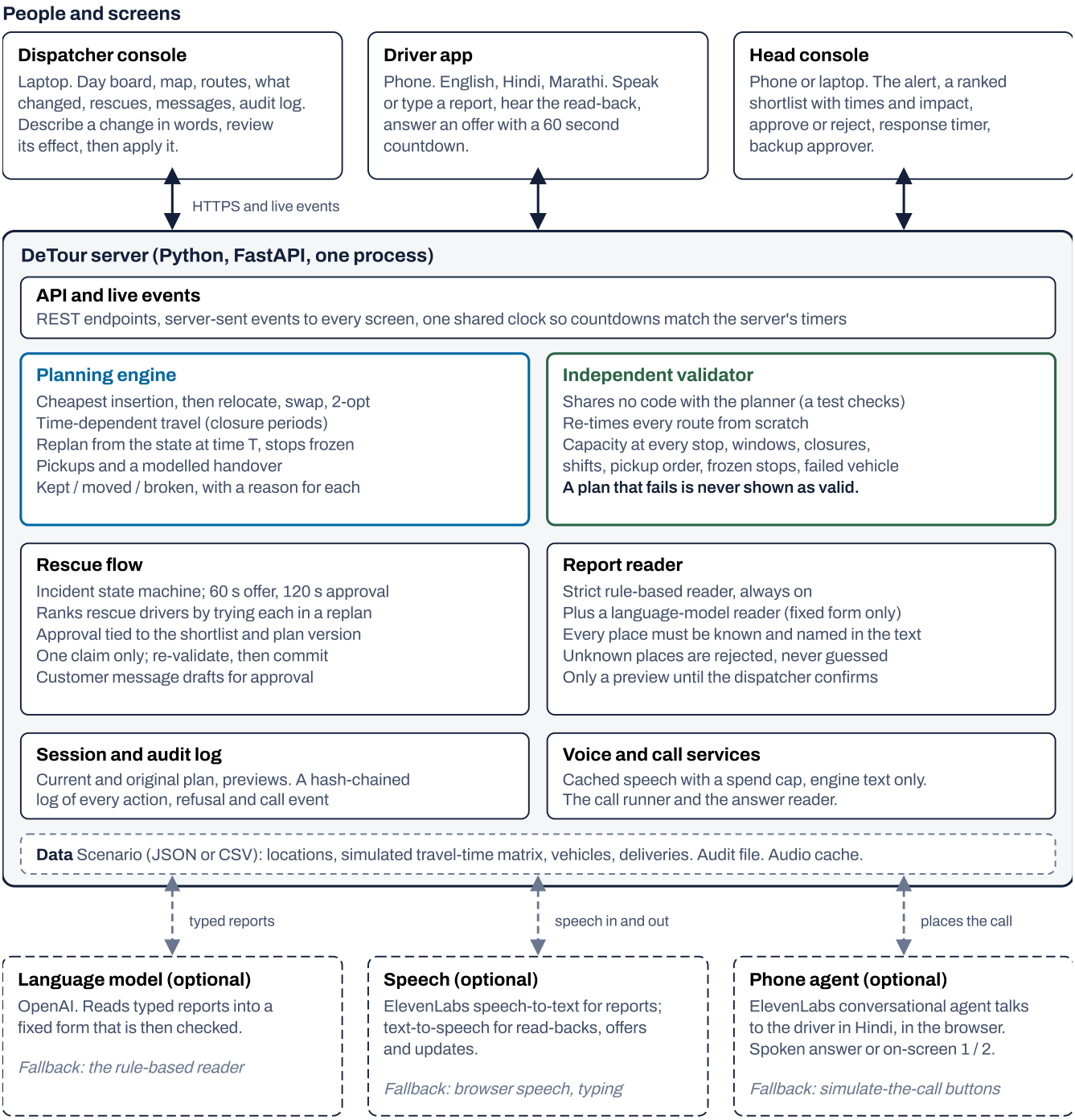


Figure 2. System architecture. The planning engine and the independent validator are the core and run with no external service. The language, speech and phone services (dashed) are optional layers, and each has a local fallback. The planner's output always passes through the validator before it is shown or committed. The server follows the voice conversation and records the driver's answer from it.

Components and technology

Part	What it is
Planning engine	Python. Typed data model, deterministic solver, time-dependent travel model, replanner with frozen history, pickup and handover modelling, and a diff that classifies every delivery.
Independent validator	A separate Python package working on plain data. An automated test fails the build if it ever imports the planner.
Server	FastAPI. REST endpoints plus a server-sent event stream, so every screen updates live. State is held in one process, with the audit log written to disk.
Screens	React and TypeScript. Dispatcher console, driver app and head console, responsive from laptop to phone. The driver app works in three languages.
AI and phone	OpenAI for typed reports (optional). ElevenLabs for speech-to-text, text-to-speech and the voice agent.
Data	Fictional only: a built-in demo scenario, or a scenario loaded from JSON or CSV. Travel times come from a simulated matrix. No live map, GPS or traffic data.
Run	One command each for the server and the screens. The core demonstration needs no paid service. Phones reach it through an HTTPS tunnel, which browsers require for the microphone.

7. Correctness and evidence

The booklet asks that decisions with real consequences follow explicit, checkable rules. We treat that as the central requirement and test it directly.

- **The validator is tested by breaking plans on purpose.** Overloaded vehicles, late arrivals, tampered times and loads, ignored closures, duplicate or missing deliveries, a failed vehicle with later stops, altered or reordered completed stops, and pickups out of order are each caught.
- **Random disruption chains.** About 18,000 random sequences of closures, breakdowns and urgent deliveries, including a rescuer breaking down while loading, were run. Every resulting plan passed the validator, history was never rewritten, and every delivery was accounted for exactly once. This found several real defects, all fixed.
- **The worked example reproduces exactly** from the engine: the ranking, the times (14:15, 14:20, 14:50 and so on), the rejection of the fourth driver, and the broken delivery.
- **The rescue rules are tested**: one claim only, late answers refused and logged, approval void on any plan change, no double-booking, honest failure when nobody accepts, and the three variants (a driver does not answer, nobody accepts, the head is silent).
- **AI is tested against its guardrails**: a model that returns a real place id the text never names is rejected, invented vehicles are refused, invalid times are refused, and an instruction hidden in a report has no effect.
- **Real services were exercised.** Speech generated by the speech service was transcribed back word for word in English, Hindi and Marathi. A recorded Marathi report went through the whole voice path in a browser. The phone agent was run against the vendor's conversation simulator for confirm, no, keypad 1, keypad 2, an unclear answer twice, and an off-topic question.

The prototype has more than 260 automated tests, run with one command for the server and one for the screens.

8. Prototype status, limits and next steps

Working now

- Planning, three disruptions, replanning from state, the independent validator, and the kept / moved / broken comparison with reasons.
- The dispatcher console, the driver app in three languages, and the head console.
- Typed reports read by a language model with rule-based checks, with dispatcher confirmation.
- Spoken reports and spoken read-backs, offers and updates, with cached audio and a spending cap.
- The full rescue flow with its timers, the escalation call logic, the audit log, and customer message drafts.

Honest limits

- The solver is a heuristic. It is not provably optimal.
- Travel times are simulated. There is no live map or traffic.
- A parcel on a running vehicle cannot move to another vehicle; a roadside handover between running vehicles is not modelled.
- State is held in one process. There is no sign-in, so the demonstration relies on an unguessable tunnel address.
- **The escalation call is a live voice conversation in the driver's browser, not a call over a phone line**, because a phone number needs a paid service. The call logic is tested against a faked line, and the agent against the vendor's simulator.
- The voice path was tested in a desktop browser, not yet on physical phones.

Next steps

- Run the live three-phone rehearsal, including the voice call, with a fallback for every step.
- Check every acceptance criterion of the specification end to end, and document assumptions and limits in the repository README.
- After the event: sign-in and roles, a database for state, and a road-network travel-time source in place of the simulated matrix.